

Loan Payoff Prediction Model using deep learning Neural Networks

By

***KEDAR. R.
PATIL***

Net ID – rk6443

***SPRING 2020 –
BAN 693***

Advisor

Prof Jiming Wu

**California State
University, East
Bay**

**Business
Analytic MS**

Index

Sr. No.	Topics	Page no.
1	Introduction	3
2	Tools Used	4
3	Research Methodology	5
4	Data Description	5
5	Data Overview	8
6	Exploratory Data Analysis	10
7	Dealing with Missing Data	12
8	Dealing with Categorical Variables and Dummy Variables	13
9	Data Preprocessing and Model selection	15
10	Neural Network model creation	16
11	Model Performance Evaluation (Conclusion)	18
12	References	20

Introduction

Lending Club is a US peer-to-peer lending company, headquartered in San Francisco, California. It was the first peer-to-peer lender to register its offerings as securities with the Securities and Exchange Commission (SEC) and to offer loan trading on a secondary market. Lending Club is the world's largest peer-to-peer lending platform. Since 2007, the lending club has been bringing borrowers and investors together, transforming the way people access to credit. Over the last 10 years, it has helped millions of people take control of their debt, grow their small businesses, and invest in the future.

I have used the subset of the Lending Club Data Set obtained from Kaggle.com. Kaggle is an Airbnb for Data Scientists. This is where they spend their nights and weekends. It's a crowd-sourced platform to attract, nurture, train, and challenge data scientists from all around the world to solve data science, machine learning and predictive analytics problems. It has over 536,000 active members from 194 countries and it receives close to 150,000 submissions per month. Started from Melbourne, Australia Kaggle moved to Silicon Valley in 2011, raised some 11 million dollars from the likes of Hal Varian (Chief Economist at Google), Max Levchin (PayPal), Index and Khosla Ventures and then ultimately been acquired by the Google in March of 2017. Kaggle is the number one stop for data science enthusiasts all around the world who compete for prizes and boost their Kaggle rankings. There are only 94 Kaggle Grandmasters in the world to this date.

Artificial intelligence refers to using computer systems to perform tasks that normally require human intelligence and can be considered as a concept broader than machine learning, even though they are often used interchangeably. Nowadays, more and more organizations are employing artificial intelligence and Big Data to gain an edge over the rest of the market. There

are ample of machine learning algorithms to predict whether a borrower will be able to pay back the loan or not. Many researchers consider machine learning as one of the most useful research methodologies to transform data into intelligent action and to capture the meaningful information and hidden patterns from the historic datasets. Deep learning (also known as deep structured learning) is part of a broader family of machine learning methods based on artificial neural networks with representation learning. Learning can be supervised, semi supervised or unsupervised. Deep learning architectures such as deep neural networks, deep belief networks, recurrent neural networks and convolutional neural networks have been applied to fields including computer vision, speech recognition, natural language processing, audio recognition, social network filtering, machine translation, bioinformatics, drug design, medical image analysis, material inspection and board game programs, where they have produced results comparable to and in some cases surpassing human expert performance.

In this paper, I have used deep learning with Python to develop prediction models to forecast whether borrowers will repay their loans or not. Given historical data on loans given out with information on whether or not the borrower defaulted (charge-off), can I build a model that can predict whether or not a borrower will pay back their loan? This way in the future when we get a new potential customer we can assess whether or not they are likely to pay back the loan.

Tools Used

Open source distribution of Python, Anaconda installed in local machine used to code in python and apply machine learning algorithm.

Anaconda is a free and open-source distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify package management and deployment. The distribution includes data-science packages suitable for Windows, Linux, and mac OS. It is developed and maintained by Anaconda, Inc., which was founded by Peter Wang and Travis Oliphant in 2012.

Research Methodology

Banking sector has grown immensely in the early 21st century. The one of the biggest part of banking sector is Loaning sector. Roughly 70% of the American population live on credit. Amongst 70% of the population, 55% of American's credit card have debt. According to Lending Tree revolving and non-revolving debt is currently at \$3.86 trillion. It predicts it will pass \$4 trillion this year (2020).

From Kaggle.com, I found a Lending club dataset about loans given out with information on whether or not the borrower defaulted (charge-off). The dataset can be downloaded from the Kaggle website at <https://www.kaggle.com/wordsforthewise/lending-club>. This is the loan borrower dataset of about 396k borrowers with different tenure, loan amount, interest rate etc. We will further study the dataset and its metrics.

Data Description

This Lending Club dataset includes 396k observations, each with 28 attributes. The target y attribute shows whether the borrower repaid the loan or not. The target feature is coded as

‘Fully paid’ to indicate borrower has fully paid the loan amount while ‘Charged off’ means the creditor has declared that the amount of debt is unlikely to be collected. The 28 attributes are described below.

Sr no.	Attributes	Description
0	loan_amnt	The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value.
1	term	The number of payments on the loan. Values are in months and can be either 36 or 60.
2	int_rate	Interest Rate on the loan
3	installment	The monthly payment owed by the borrower if the loan originates.
4	grade	LC assigned loan grade
5	sub_grade	LC assigned loan subgrade
6	emp_title	The job title supplied by the Borrower when applying for the loan.*
7	emp_length	Employment length in years. Possible values are between 0 and 10 where 0 means less than one year and 10 means ten or more years.
8	home_ownership	The home ownership status provided by the borrower during registration or obtained from the credit report. Our values are: RENT, OWN, MORTGAGE, OTHER
9	annual_inc	The self-reported annual income provided by the borrower during registration.

10	verification_status	Indicates if income was verified by LC, not verified, or if the income source was verified
11	issue_d	The month which the loan was funded
12	loan_status	Current status of the loan
13	purpose	A category provided by the borrower for the loan request.
14	title	The loan title provided by the borrower
15	zip_code	The first 3 numbers of the zip code provided by the borrower in the loan application.
16	addr_state	The state provided by the borrower in the loan application
17	dti	A ratio calculated using the borrower's total monthly debt payments on the total debt obligations, excluding mortgage and the requested LC loan, divided by the borrower's self-reported monthly income.
18	earliest_cr_line	The month the borrower's earliest reported credit line was opened
19	open_acc	The number of open credit lines in the borrower's credit file.
20	pub_rec	Number of derogatory public records
21	revol_bal	Total credit revolving balance
22	revol_util	Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit.
23	total_acc	The total number of credit lines currently in the borrower's credit file
24	initial_list_status	The initial listing status of the loan. Possible values are – W, F

25	application_type	Indicates whether the loan is an individual application or a joint application with two co-borrowers
26	mort_acc	Number of mortgage accounts.
27	pub_rec_bankruptcies	Number of public record bankruptcies

Data Overview

Table 1: Datatypes of attributes

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):
loan_amnt          396030 non-null float64
term               396030 non-null object
int_rate           396030 non-null float64
installment        396030 non-null float64
grade              396030 non-null object
sub_grade          396030 non-null object
emp_title          373103 non-null object
emp_length         377729 non-null object
home_ownership     396030 non-null object
annual_inc         396030 non-null float64
verification_status 396030 non-null object
issue_d            396030 non-null object
loan_status        396030 non-null object
purpose            396030 non-null object
title              394275 non-null object
dti                396030 non-null float64
earliest_cr_line   396030 non-null object
open_acc           396030 non-null float64
pub_rec            396030 non-null float64
revol_bal          396030 non-null float64
revol_util         395754 non-null float64
total_acc          396030 non-null float64
initial_list_status 396030 non-null object
application_type    396030 non-null object
mort_acc           358235 non-null float64
pub_rec_bankruptcies 395495 non-null float64
address            396030 non-null object
dtypes: float64(12), object(15)
memory usage: 81.6+ MB
```

Table 2: Data Frame summary

```

df.describe

<bound method NDFrame.describe of
0      10000.0  36 months  11.44    329.48    B    B4
1       8000.0  36 months  11.99    265.68    B    B5
2      15600.0  36 months  10.49    506.97    B    B3
3       7200.0  36 months   6.49    220.65    A    A2
4      24375.0  60 months  17.27    609.33    C    C5
...
396025  10000.0  60 months  10.99    217.38    B    B4
396026  21000.0  36 months  12.29    700.42    C    C1
396027   5000.0  36 months   9.99    161.32    B    B1
396028  21000.0  60 months  15.31    503.02    C    C2
396029   2000.0  36 months  13.61     67.98    C    C2

      emp_title  emp_length  home_ownership  annual_inc  ... \
0      Marketing    10+ years          RENT    117000.0  ... \
1  Credit analyst     4 years    MORTGAGE    65000.0  ...
2  Statistician    < 1 year          RENT    43057.0  ...
3  Client Advocate     6 years          RENT    54000.0  ...
4  Destiny Management Inc.     9 years    MORTGAGE    55000.0  ...
...
396025  licensed bankere     2 years          RENT    40000.0  ...
396026      Agent     5 years    MORTGAGE    110000.0  ...
396027      City Carrier    10+ years          RENT    56500.0  ...
396028  Gracon Services, Inc    10+ years    MORTGAGE    64000.0  ...
396029  Internal Revenue Service    10+ years          RENT    42996.0  ...

      open_acc  pub_rec  revol_bal  revol_util  total_acc  initial_list_status  \
0         16.0     0.0    36369.0     41.8     25.0                w
1         17.0     0.0    20131.0     53.3     27.0                f
2         13.0     0.0    11987.0     92.2     26.0                f
3          6.0     0.0     5472.0     21.5     13.0                f
4         13.0     0.0    24584.0     69.8     43.0                f
...
396025     6.0     0.0     1990.0     34.3     23.0                w
396026     6.0     0.0     43263.0     95.7      8.0                f
396027    15.0     0.0     32704.0     66.9     23.0                f
396028     9.0     0.0     15704.0     53.8     20.0                f
396029     3.0     0.0     4292.0     91.3     19.0                f

      application_type  mort_acc  pub_rec_bankruptcies  \
0      INDIVIDUAL         0.0             0.0
1      INDIVIDUAL         3.0             0.0
2      INDIVIDUAL         0.0             0.0
3      INDIVIDUAL         0.0             0.0
4      INDIVIDUAL         1.0             0.0
...
396025  INDIVIDUAL         0.0             0.0
396026  INDIVIDUAL         1.0             0.0
396027  INDIVIDUAL         0.0             0.0
396028  INDIVIDUAL         5.0             0.0
396029  INDIVIDUAL        NaN             0.0

      address
0      0174 Michelle Gateway\nMendozaberg, OK 22690
1      1076 Carney Fort Apt. 347\nLoganmouth, SD 05113
2      87025 Mark Dale Apt. 269\nNew Sabrina, WV 05113
3      823 Reid Ford\nDelacruzside, MA 00013
4      679 Luna Roads\nGreggshire, VA 11650
...
396025  12951 Williams Crossing\nJohnnyville, DC 30723
396026  0114 Fowler Field Suite 028\nRachelborough, LA...
396027  953 Matthew Points Suite 414\nReedfort, NY 70466
396028  7843 Blake Freeway Apt. 229\nNew Michael, FL 2...
396029  787 Michelle Causeway\nBriannaton, AR 48052

[396030 rows x 27 columns]>

```

Exploratory Data Analysis

Exploratory Data analysis is a very important step in Machine learning. We need to check how the data is? Which attributes and features are contributing to the dataset and how we need to enhance or optimize it to make the model better.

Table 1: Count Plot of 'Loan Status'

It's always a good idea to do a count plot in classification problem to explore the actual balancing of the labels of the feature in Data set

```
sns.countplot(data = df,x = 'loan_status')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1673ddb1188>
```

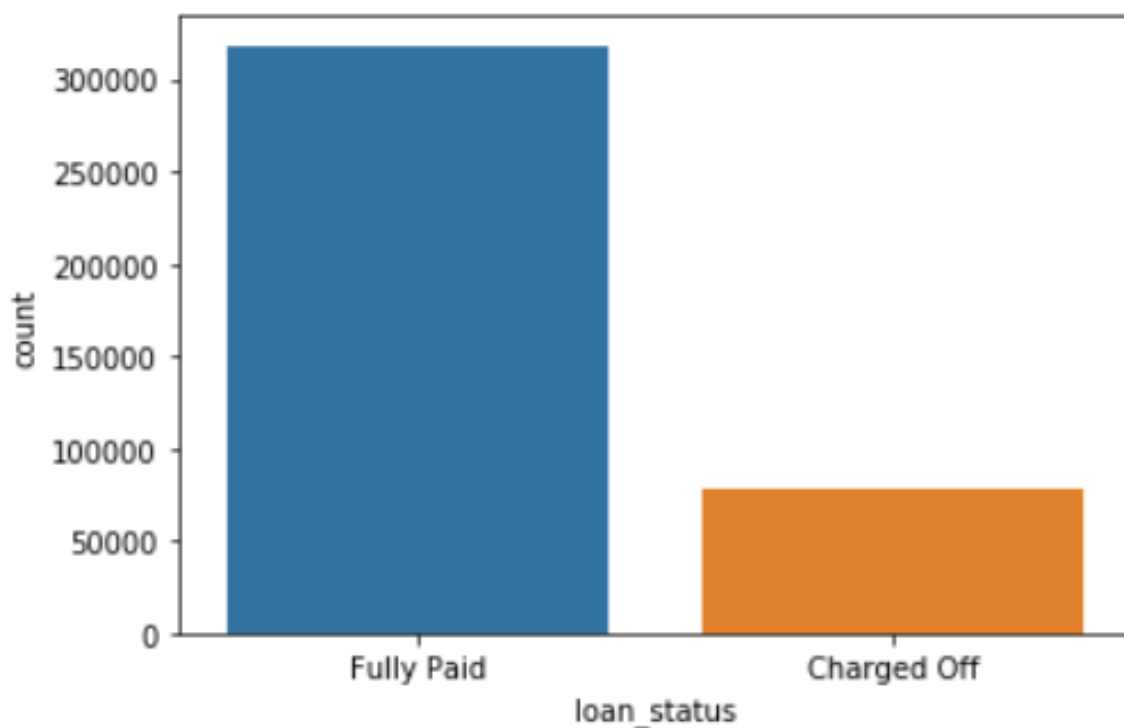
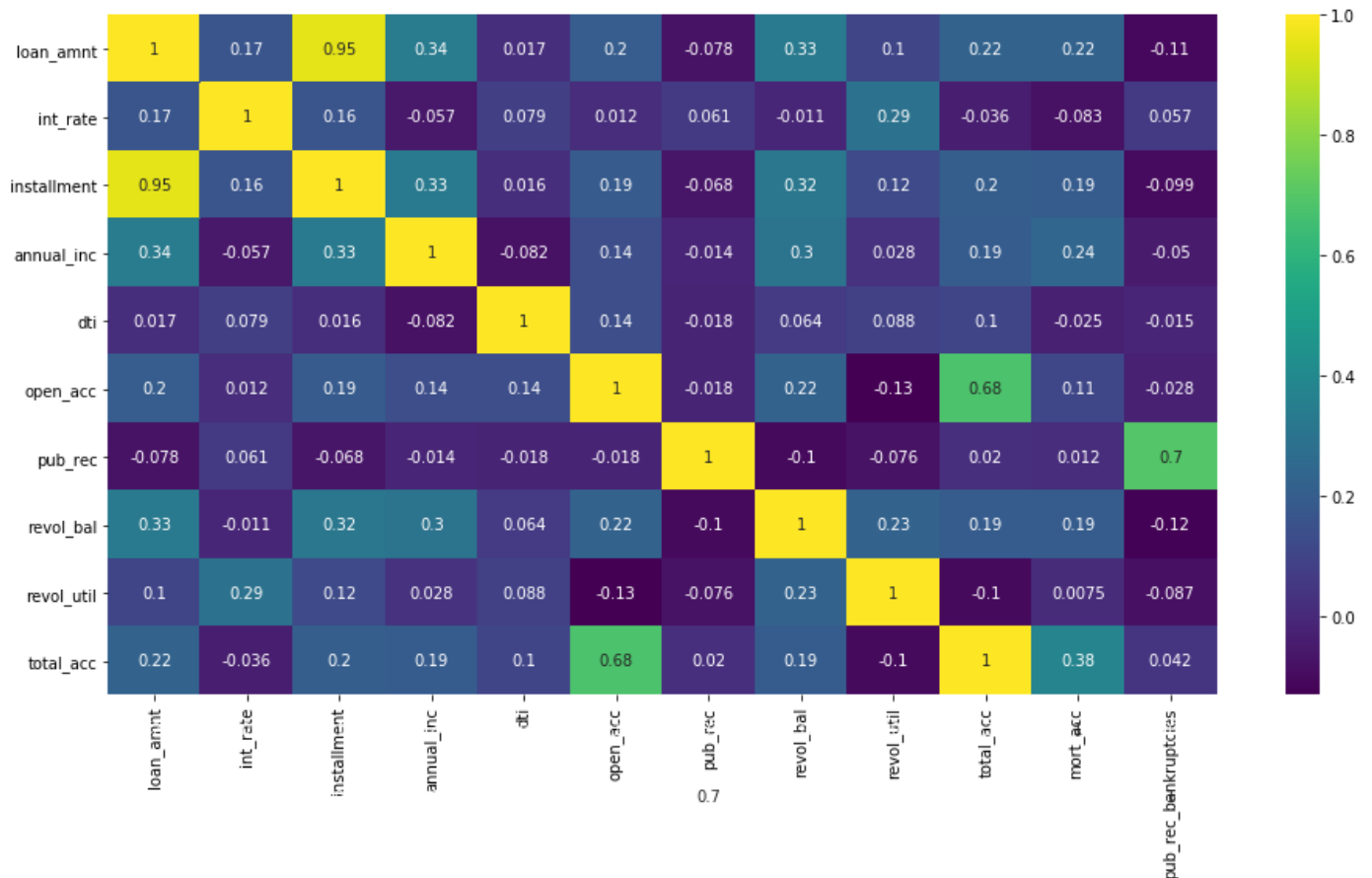


Table 2: Heat map of correlation between the variables

```
In [12]: plt.figure(figsize=(16,8))
sns.heatmap(data= df.corr(),annot = True,cmap = 'viridis')
plt.ylim(10,0)
```

Out[12]: (10, 0)



The figure above is the Heat map of the correlation between all the attributes of our dataset. For some attributes I have got very good correlation and for some attributes are not that well correlated. Let's discuss about 'loan_amnt' attribute, it is highly correlated with 'installment'.

Reason being that the loan_amnt highly correlated with 'installment' is that both the attributes go hand in hand. Installments are dependent on loan amount. If loan about is more, then installments are more and vice versa. The attributes which are moderately high correlated are 'open_acc' and 'total_acc'.

‘open_acc’ → the number of open credit lines in the borrower's credit file.

‘total_acc’ → the total number of credit lines currently in the borrower's credit file.

It perfectly makes sense that why these two attributes are highly correlated. Total credit line accounts acts as the superset and open credit line accounts is its subset. It depends on the borrower's account that how many open credit line accounts are still open for him/her.

The next highly correlated attributes are ‘pub_rec’ and ‘pub_rec_bankruptcies’.

‘pub_rec’ → Number of derogatory public records.

‘pub_rec_bankruptcies’ → Number of public record bankruptcies.

Public record bankruptcies acts as a superset which has all the records and public records acts as its subset. It depends on the borrower's account that how many total derogatory public records are there amongst total number of public record bankruptcies. Hence it's highly correlated.

Dealing with Missing Data

Missing values are representative of the messiness of real world data. There can be a lot

```
df.isnull().sum() / len(df) * 100
```

loan_amnt	0.000000
term	0.000000
int_rate	0.000000
installment	0.000000
grade	0.000000
sub_grade	0.000000
emp_title	5.789208
emp_length	4.621115
home_ownership	0.000000
annual_inc	0.000000
verification_status	0.000000
issue_d	0.000000
loan_status	0.000000
purpose	0.000000
title	0.443148
dti	0.000000
earliest_cr_line	0.000000
open_acc	0.000000
pub_rec	0.000000
revol_bal	0.000000
revol_util	0.069692
total_acc	0.000000
initial_list_status	0.000000
application_type	0.000000
mort_acc	9.543469
pub_rec_bankruptcies	0.135091
address	0.000000
loan_repaid	0.000000
dtype:	float64

of reasons why they occur: ranging from human errors during data entry, incorrect readings or data parsing to software bugs in the data processing pipeline. After exploring the data, I got to know there are missing data in the dataset. The absence of data reduces the statistical power which refers to the probability that the test will reject the null hypothesis when it is false. Also, the lost data can cause bias in the estimation of parameters. Hence it is very important to remove it. After data analysis, I found out that the some of

the attributes have missing data. The percent of missing data compared to actual dataset is shown here. After analysis, the 'emp_title' column was making a significance difference to dataset as many employee has unique title and it's not informative. Hence I dropped the entire column from the Data frame. I dropped 'emp_length' from the data frame. After analysis and visualizing the values, I found out that values are extremely similar all other employment years, hence dropped it. I dropped the 'title' column because it's not providing any significant information. The 'mort_acc' column is the no. of mortgage accounts. The columns has almost 9.5% of the data of dataset, hence we can't drop the entire column. I checked the correlation of other columns with mortgage account column. The 'total_acc' is fairly correlated with 'mort_acc'. I filled the missing data in mortgage account with the data in total account using user defined and lambda function in python. I dropped 'revol_util' and 'pun_rec_bankruptcies' columns as the missing data was significantly very small.

Dealing with Categorical Variables and Dummy Variables

In a Dataset we have lots of categorical variables and it should be in a proper way to demonstrate our data otherwise we might get dispersed or wrong interpreted data.

In the Figure shown below. I have check the columns with string values only. Now we have to convert the following string valued columns into numeric in order to train it.

```

In [84]: #listing all columns which are non numeric values|
df.select_dtypes(['object']).columns

Out[84]: Index(['term', 'grade', 'sub_grade', 'home_ownership', 'verification_status',
               'issue_d', 'loan_status', 'purpose', 'earliest_cr_line',
               'initial_list_status', 'application_type', 'address'],
              dtype='object')
```

```
In [86]: df['term'].value_counts()
```

```
Out[86]: 36 months    301247
        60 months    93972
        Name: term, dtype: int64
```

```
In [88]: df['term'] = df['term'].apply(lambda term: int(term[:3]))
```

```
In [90]: df['term'].value_counts()
```

```
Out[90]: 36    301247
        60    93972
        Name: term, dtype: int64
```

Here I am dealing with
'term' column. It is a
categorical variable and has
values in two category: 36
months and 60 months.
Hence using lambda

function. I changed the data type of the column to 36 and 60 ie. An integer. The 'sub_grade' is the part of grade. When we call 'sub_grade', we eventually get details the of 'grade' column as well. For example, if we want to call A1, A2, A3 etc. then we already know it's the part of 'A' grade from the Grade column. Converting grade column values into dummy variables wouldn't make any significant difference to the dataset as we already have 'sub_grade' hence I dropped this column.

```
dummies = pd.get_dummies(df[['verification_status', 'application_type', 'initial_list_status', 'purpose']], drop_first=True)
df = pd.concat([df.drop(['verification_status', 'application_type', 'initial_list_status', 'purpose'], axis=1), dummies], axis=1)
```

```
dummies = pd.get_dummies(df['sub_grade'], drop_first=True)
#Drop_first = true because is there A/B/C, if we drop A and B then answer is C.
df = pd.concat([df.drop('sub_grade', axis=1), dummies], axis=1)
```

The 'subgrade' column is string hence I dealt with it using dummy variables. I changed all the grades like 'A2', 'A3'... 'G5'.

```
: df.columns
```

```
: Index(['loan_amnt', 'term', 'int_rate', 'installment', 'home_ownership',
        'annual_inc', 'issue_d', 'loan_status', 'dti', 'earliest_cr_line',
        'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc',
        'mort_acc', 'pub_rec_bankruptcies', 'address', 'loan_repaid', 'A2',
        'A3', 'A4', 'A5', 'B1', 'B2', 'B3', 'B4', 'B5', 'C1', 'C2', 'C3', 'C4',
        'C5', 'D1', 'D2', 'D3', 'D4', 'D5', 'E1', 'E2', 'E3', 'E4', 'E5', 'F1',
        'F2', 'F3', 'F4', 'F5', 'G1', 'G2', 'G3', 'G4', 'G5',
        'verification_status_Source Verified', 'verification_status_Verified',
        'application_type_INDIVIDUAL', 'application_type_JOINT',
        'initial_list_status_w', 'purpose_credit_card',
        'purpose_debt_consolidation', 'purpose_educational',
        'purpose_home_improvement', 'purpose_house', 'purpose_major_purchase',
        'purpose_medical', 'purpose_moving', 'purpose_other',
        'purpose_renewable_energy', 'purpose_small_business',
        'purpose_vacation', 'purpose_wedding'],
        dtype='object')
```

In this figure, we can see that I
have assigned different dummy
variables to the original data of the
'subgrade' column. In the same

way I dealt with other categorical variables.

Data Preprocessing and Model selection

After completing with exploratory data analysis and data cleaning, now is the time for Data preprocessing.

Train Test Split

```
: from sklearn.model_selection import train_test_split
: df = df.drop('loan_status',axis = 1)
: X = df.drop('loan_repaid',axis = 1).values
: y = df['loan_repaid'].values
: # df = df.sample(frac=0.1,random_state=101)
: print(len(df))
395219
: X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=101)
```

Here I used Sklearn to import the train test split. The y variable is the 'loan_repaid' column, this is my dependent variable and rest of the columns are in X, these are my independent

variables. I used train test and split at 20% testing data and random state as 101.

Normalizing the Data

```
1]: from sklearn.preprocessing import MinMaxScaler
2]: scaler = MinMaxScaler()
3]: X_train = scaler.fit_transform(X_train)
4]: X_test = scaler.transform(X_test)
```

A data normalization definition isn't hard to find, but settling on a specific one can be a bit tricky. Taking into account all the different explanations out there, data normalization is essentially a type of process

wherein data within a database is reorganized in such a way so that users can properly utilize that database for further queries and analysis. There are some goals in mind when undertaking the data normalization process. The first one is to get rid of any duplicate data that might appear within the data set. This basically goes through the database and eliminates any redundancies

that may occur. Redundancies can adversely affect analysis of data since they are values which aren't exactly needed. Expunging them from the database helps to clean up the data, making it easier to analyze. The other goal is to logically group data together. You want data that relates to each other to be stored together. This will occur in a database which has undergone data normalization. If data is dependent on each other, they should be in close proximity within the data set. Hence I normalized the data.

Neural Network model creation

As we discussed Deep learning, I want to shed light on deep learning's most popular application i.e. artificial neural networks. I have used ANN in this project for predict whether borrower will repay the loan or not. Artificial Neural Networks are the pieces of a computing system designed to simulate the way the human brain analyzes and processes information. They are the foundations of Artificial Intelligence (AI) and solve problems that would prove impossible or difficult by human or statistical standards. ANN have self-learning capabilities that enable them to produce better results as more data become available. Artificial Neural Networks (ANN) are paving the way for life-changing applications to be developed for use in all sectors of the economy. Artificial Intelligence (AI) platforms that are built on ANN are disrupting the traditional way of doing things.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout
```

```
model = Sequential()
model.add(Dense(78,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(39,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(19,activation = 'relu'))
model.add(Dropout(0.2))

model.add(Dense(units = 1,activation = 'sigmoid'))

model.compile (loss = 'binary_crossentropy',optimizer = 'adam')
```

```
X_train.shape
(316175, 78)
```

Here I imported tensorflow's sequential model and dense and dropout layers. I checked the shape of X_train and I got to know there 78 features in it. Hence I selected first dense layer with 78 layers and activation function as rectified linear unit.

To overcome overfitting I added dropout layer with 20% dropout rate. Also, I added hidden layers with half of each layers and added dropouts to each layer as well. There is an output layer as well which acts as an end dense neuron with sigmoid activation function as it pushes values between zero and one. I compiled the model with binary cross entropy classification and 'adam' optimizer.

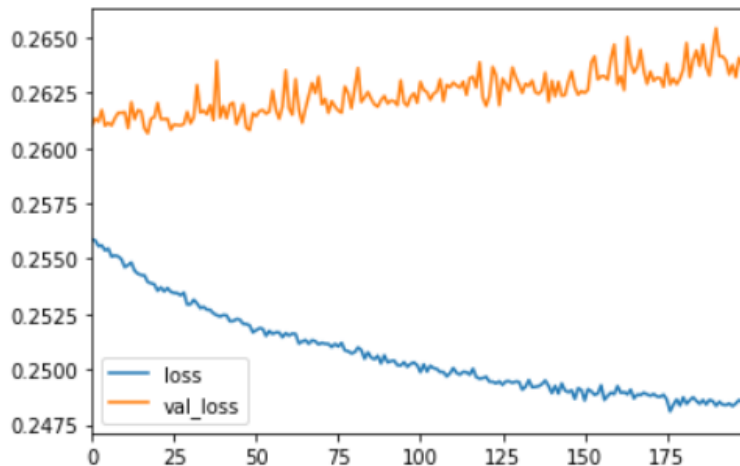
```
model.fit(x = X_train,y = y_train,epochs = 200, batch_size = 256, validation_data = (X_test,y_test))
Epoch 192/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2485 - val_loss: 0.2641
Epoch 193/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2484 - val_loss: 0.2641
Epoch 194/200
316175/316175 [=====] - 4s 14us/sample - loss: 0.2484 - val_loss: 0.2640
Epoch 195/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2485 - val_loss: 0.2633
Epoch 196/200
316175/316175 [=====] - 4s 13us/sample - loss: 0.2483 - val_loss: 0.2638
Epoch 197/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2485 - val_loss: 0.2632
Epoch 198/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2486 - val_loss: 0.2641
Epoch 199/200
316175/316175 [=====] - 4s 14us/sample - loss: 0.2486 - val_loss: 0.2637
Epoch 200/200
316175/316175 [=====] - 4s 12us/sample - loss: 0.2480 - val_loss: 0.2639
<tensorflow.python.keras.callbacks.History at 0x167033bab48>
```

I fit the model with 200 epochs, batch size as 256 as dataset size is quite large and added validation data to get the data or losses.

After fitting the model, now it's the time to evaluate it. I have retrieved the losses and plotted it as well.

```
losses.plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x16701a37b88>
```



After plotting the losses, we can see that validation loss is in orange and training loss is in blue. Validation loss here is much greater than training loss and we can

infer from this that model is not overfitting and it's very well fitted model.

Model Performance Evaluation (Conclusion)

In this project, we explored deep learning neural network model in depth and also learnt that how effective it is by exploring real dataset of Lending Club. After model creation, I evaluated the model considering the trained and tested data.

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
predictions = model.predict_classes(X_test)
```

```
print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.96	0.45	0.61	15658
1	0.88	0.99	0.93	63386
accuracy			0.89	79044
macro avg	0.92	0.72	0.77	79044
weighted avg	0.90	0.89	0.87	79044

After importing classification report, we can see that the model has 89% accuracy. In real world 89% accuracy is considered a very good

accuracy. We got a really great precision of 96% for the model as well. Overall conclusion from the classification report is that we can say model is learning something from the dataset.

I imported confusion matrix from the metrics. Figure shows how much the data is misclassified.

```
!]: print(confusion_matrix(y_test,predictions))
```

Calculating the misclassified data →

```
[[ 7080  8578]
 [  319 63067]]
```

$(8578 + 319) / (8578 + 319 + 7080 + 63067) = 0.1125$ i.e. 11.25%. We can

say that the model has 11.25% misclassification rate. In real world, this misclassification rate is very moderate and we can say that our Deep learning neural networks model is 89% accurate in predicting the Data.

To cross verify the classification report and confusion matrix, I have created a random customer to check whether he would repay the loan or not.

```
: import random
random.seed(101)
random_ind = random.randint(0,len(df))

new_customer = df.drop('loan_repaid',axis=1).iloc[random_ind]
new_customer

: loan_amnt      25000.00
term             60.00
int_rate         18.24
installment      638.11
annual_inc       61665.00
...
30723             1.00
48052             0.00
70466             0.00
86630             0.00
93700             0.00
Name: 305323, Length: 78, dtype: float64

: new_customer = scaler.transform(new_customer.values.reshape(1,78))

: model.predict_classes(new_customer)

: array([[1]])

: df.iloc[random_ind]['loan_repaid']

: 1.0
```

I imported random seed at 101 and using random_ind I grabbed a random new customer. I got the data of random customer and scaled it using scaler transform. After predicting I got output as array [1]. Later on I check whether loan was repaid or not, and it

was repaid. In conclusion, we can say that the model predicted correctly that the new customer will repay the loan amount.

References

- [1] https://en.wikipedia.org/wiki/Artificial_neural_network
- [2] <http://news.mit.edu/2017/explained-neural-networks-deep-learning-0414>
- [3] https://scholar.google.com/scholar?q=neural+network+machine+learning&hl=en&as_sdt=0&as_vis=1&oi=scholar
- [4] https://www.coursera.org/learn/machine-learning?utm_source=gg&utm_medium=sem&utm_content=07-StanfordML-US&campaignid=685340575&adgroupid=35524166541&device=c&keyword=machine%20learning%20certification%20programs&matchtype=b&network=g&devicemodel=&adposition=&creativeid=243289762922&hide_mobile_promo&gclid=CjwKCAjw4871BRAjEiwAbxXi29Kd5pgbEW4iHOE7_SdL7aryOw8dX1rrQ0qi3KsgP1Bpw11qY65zdBoCVicQAvD_BwE
- [5] <https://towardsdatascience.com/intro-to-deep-learning-c025efd92535>
- [6] <https://www.mathworks.com/discovery/deep-learning.html>
- [7] <https://www.geeksforgeeks.org/introduction-deep-learning/>